STRUMPACK meets Kernel Matrices

Elizaveta Rebrova summer internship report

Hosting Site: Lawrence Berkeley National Laboratory, Scalable Solvers Group
Mentor(s): Xiaoye Sherry Li

Abstract

As the main part of my internship I investigated ways to apply STRUMPACK algorithm (fast linear solver developed by Scalable Solvers Group) to kernel matrices (coming from machine learning and non-parametric statistics applications). Effective preprocessing methods allowed up to 10 times more efficient compression of the matrix than naive application of STRUMPACK. I have also tested compressed matrices on machine learning algorithms to confirm that — with the correct hyperparameter tuning — they do not lose prediction capability of the full (not compressed) kernel matrix. Additionally, I led about 16 meetings of the machine learning seminar for the group members, where we have discussed mathematical and computational ideas behind the major machine learning concepts and algorithms.

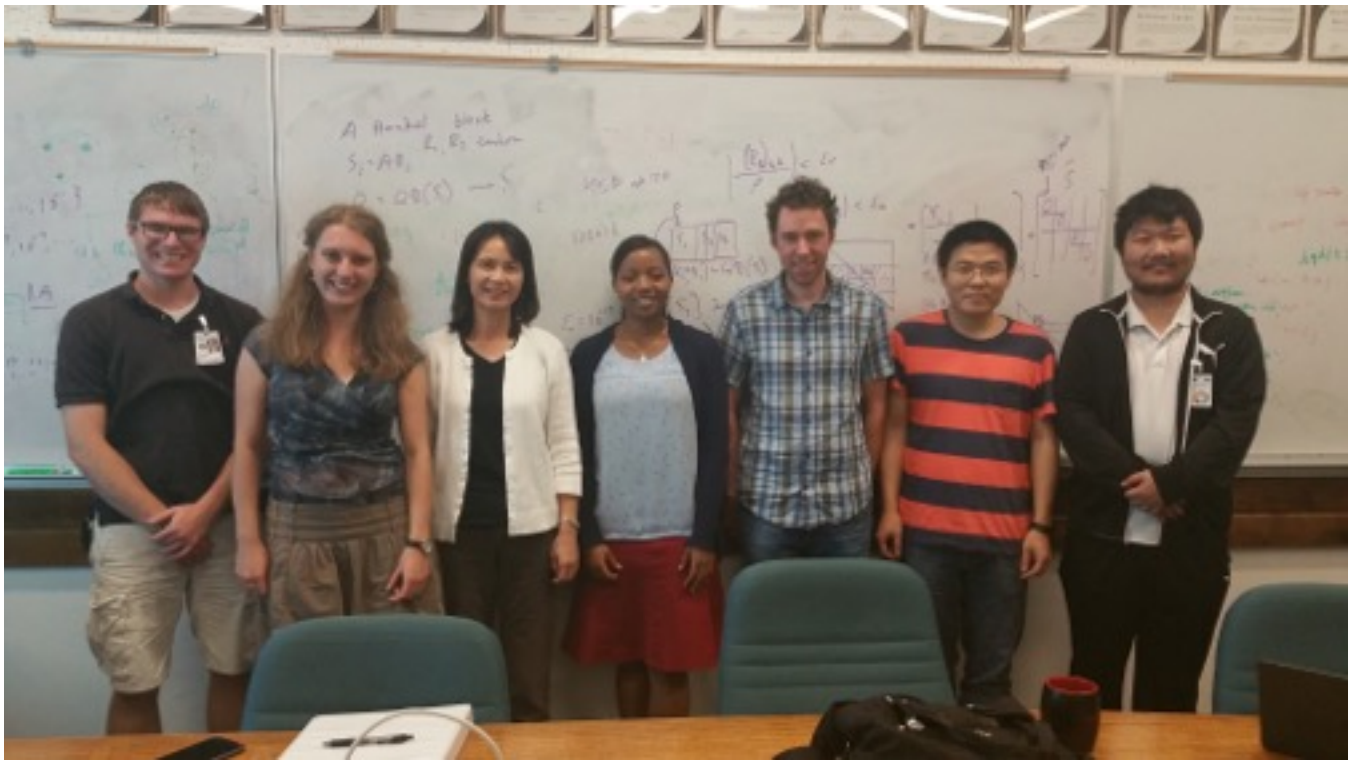

Photo of the group after our last machine learning seminar meeting. Left to right:
Christopher Gorman (summer intern), me, Xiaoye Sherri Li (senior scientist, group lead and my mentor), Juliette Ugirumurera (research post-doc), Pieter Ghysels (research scientist), Yang Liu (research post-doc), Xinliang Wang (summer intern) and Gustavo Chávez (post-doc taking this photo)

1.Internship Project

1-1. STRUMPACK algorithm for the new class of matrices

STRUMPACK - STRUctured Matrices PACKage - is a fast linear solvers and preconditioner for dense or sparse systems that uses low-rank structured factorization with randomized sampling. The package is developed over the last 3 years by the members of Scalable Solvers Group at Lawrence Berkeley National Laboratory (Xiaoye S. Li (PI), Pieter Ghysels, François-Henry Rouet and Christopher Gorman), see also [1], [2], [3]. Using STRUMPACK, one can do matrix operations (such as multiplication, factorization, inversion etc) and do them fast and in parallel. In particular, the algorithm has online version, so that one does not need to store in memory the whole matrix. One of the most powerful features of the method is that the matrix does not have to be sparse or low rank, but it should be HSS (hierarshically semi-separable), i.e. should have off diagonal sub-blocks with low rank.

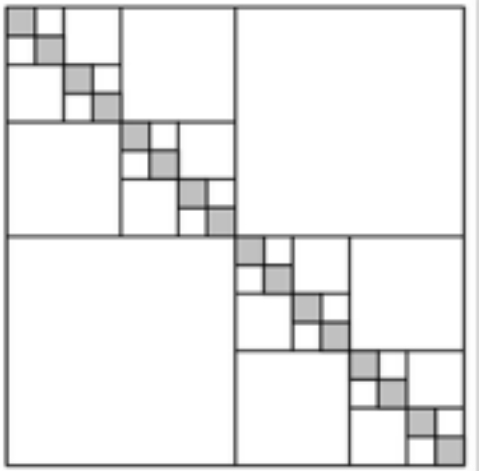

Figure 1. Matrix susceptible for STRUMPACK: white sub-matrices are assumed to be low-rank and hence can be stored efficiently, and grey sub-matrices might be dense and have to be stored as they are, but they have small size

Applications of the STRUMPACK method include the matrices coming from integral equations, boundary element method, statistics, acoustic and electromagnetic scattering theory, rational interpolation, some general discretized PDEs etc Usually HSS (off-diagonal low rank sub-blocks) property follows from the physical properties of the matrix model. For example, finite difference method for the heat equation would produce the 3-diagonal matrix due to the scheme used: it includes only the interactions between neighbor points and only elements with the indices that differ by one will

be non-zero. Hence, in the notations of Figure 1, we can take grey sub-matrices to have width two, and all white sub-matrices will be exactly zero.

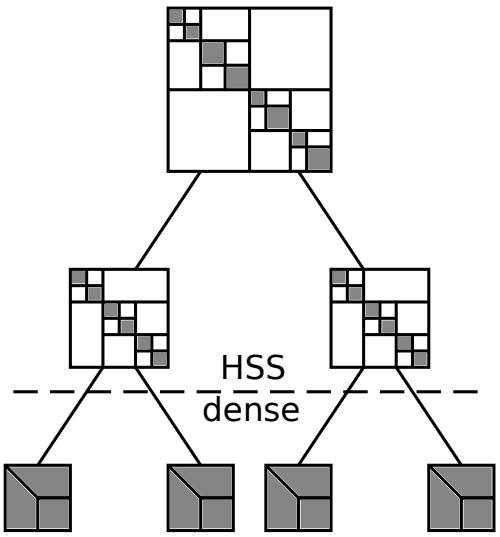

Figure 2. HSS (hierarchical semi-separable structure of embedded sub-matrices) structure of the matrix is the key to compression by STRUMPACK

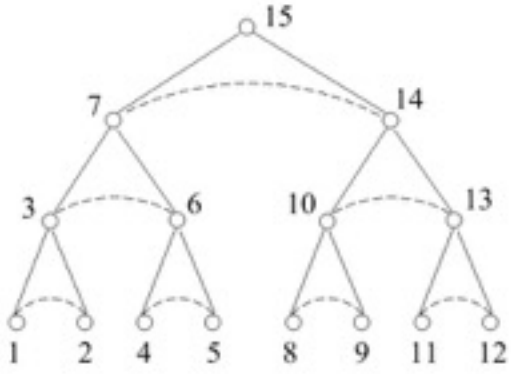

Figure 3. The algorithm builds the HSS tree of sub-matrix dependencies and uses it both for compression and for performing further operations with the compressed matrix

During my Summer internship at LBNL I had a chance to look into a structurally different class of matrices, for which STRUMPACK shows itself very useful. What if we have a matrix with lots of near zero elements and expected to have low rank sub-blocks (i.e. should have HSS structure - see Figure 2), but the positions of the small elements are unknown? This is the case with the *kernel matrices* — powerful actors of several machine learning and non-parametric statistics applications.

Intuitively, kernel matrices can be viewed as similarity matrices $K = (K_{ij})$, $i,j = 1 \ldots n$, where

$$K_{ij} = \text{similarity score between } x_i \text{—} x_j \quad (1)$$

Such K is always a square symmetric matrix, defined by a set of objects $x_1, \ldots x_n$. These objects can be anything starting from two integers, two real valued vectors, trees, words etc, provided that we know

how to compare them. Kernel matrices are used to improve and speed up the algorithms classifying these objects (see [7] for one of the easiest kernel machine learning algorithms). The task of solving linear system with a kernel matrix is a common computational bottleneck in the training phase of these algorithms.

One of the most important example of kernel matrices in machine learning applications is Gaussian kernel:

$$K_{ij} = \exp(-||x_i - x_j||^2/2h^2), \quad (2)$$

where h is gaussian width, reweighing coefficient. Here $x_1, \ldots, x_n$ are data points in d dimensional space R^d, and they are ‘’similar’’ (in sense of the definition (1) of the kernel matrix) if they are close to each other in Euclidian distance. Indeed, we can see from (2) that K_{ij} get very small very fast as long as the distance between x_i and x_j gets larger.

This last property *“large distance between data points means very small corresponding entry in the matrix K”* is the most crucial. It makes K a good candidate to be compressed by STRUMPACK (as

- K contains many near zero elements
- K contains many similar elements: formula (2) accounts only for the distances between the points, not their positions, and the distances easily can be almost the same for many pairs of points
- but K is usually full rank, having ones on diagonal and small elements otherwise)

It also suggests the main challenge associated with the new application of STRUMPACK: unlike all previous examples, we cannot assume that the dense part of the matrix K is located near its diagonal. To make STRUMPACK work efficiently, we would like to have near zero elements K_{ij} as long as $i << j$ or $j << i$, namely, that x_i is located far from all x_j when index i is far from index j.

1-2. STRUMPACK preprocessing challenge: clustering can help

Luckily, in linear algebra applications of STRUMPACK (for example, solving linear systems), we can exchange some rows and columns of the matrix in the way we want. For the data points $x_1, \ldots, x_n$ if it just means renumeration of their indices, which is harmless.

This motivates the idea behind the first part of my project: we can do much better with STRUMPACK quality with the following preprocessing:

1. we find the groups of points with large between-group distances and small within-group distances

2. and then we permute rows and columns of the matrix K so that the points of each group occupy consecutive indices (so they will form dense diagonal blocks)

Step 2 is purely technical, but step 1 suggests it is a *clustering problem.*

The task to find clusters of the points close to each other is widely studied. As Figure 4 illustrates, there are many clustering algorithms and there are no the best one: different algorithms are preferable in

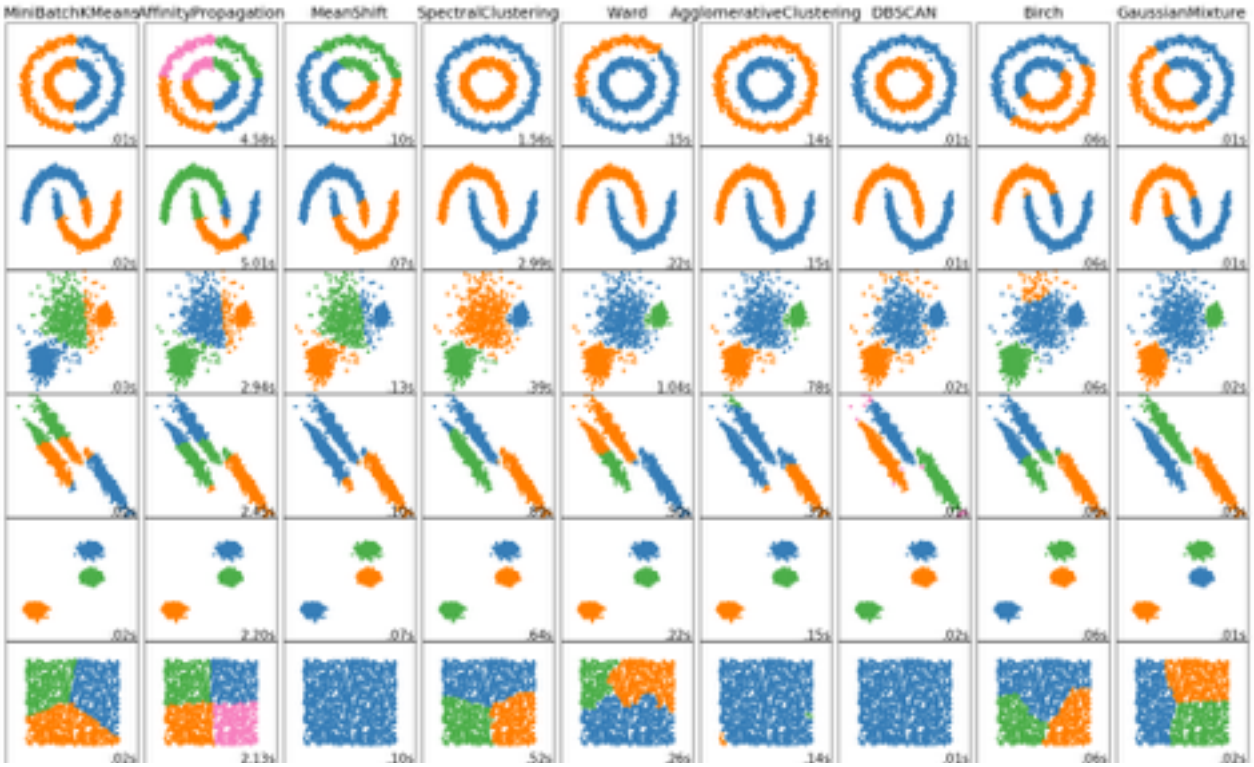

Figure 4. Variety of clustering algorithms, not equally good for various tasks (picture is taken from python documentation)

different applications. Our application also gives specific requirements for ‘’the best clustering’’, namely:

- Clusters should be small enough (otherwise dense HSS-blocks use too much memory)

- Clusters should have similar size

- The algorithm should be able to work in parallel, without storing the whole matrix, and relatively fast

- We need to construct the whole hierarchical tree of embedded clusters (corresponding to the HSS tree)

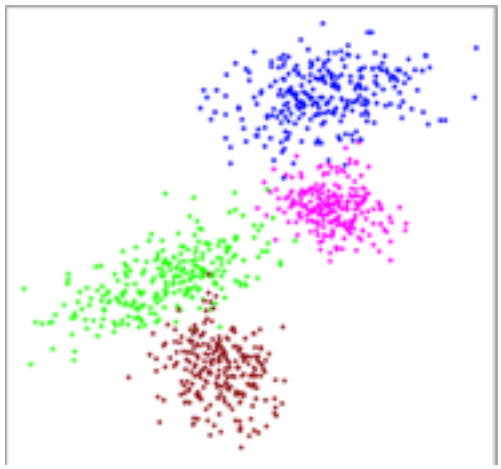

Figure 5. Construction of the tree is more involved than just clustering: having four colored clusters, which pairs should be combined first on the lower level of the HSS tree for better compression?

1-3. Some experimental results: comparison of preprocessing techniques

I have chosen and tried 5 different clustering techniques (and about 15 variations of them), both divisive and agglomerative, on about 10 different datasets. These experiments were performed on Matlab, as it allows to test clustering techniques using high-level machine learning library functions. The numbers on the Figure 6 below are also referring to the Matlab realization. STRUMPACK package is written in C++ using OpenMP and MPI parallelism. The best preprocessing methods are implemented on the ‘’full speed’’ C++ code and similar memory and quality trends are confirmed.

Figure 6 shows a piece of the comparison results obtained and it is followed by the explanation of metrics and notations used. For the observed conclusions see the end of this section.

a) Datasets

SUSY (4500, 8)	natural tree	rec 2 means	kd tree	hier (8 vertices)	hier (29 vertices)	h = 10
memory MB	6	**3.1**	3.48	23.5	9.6	
memory %	3.7	1.9	2.15	14.5	6	
max rank	60	**42**	54	50	52	
compression	1.20E-04	1.20E-04	1.30E-04	**7.70E-05**	1.10E-04	

PENDIGITS (3498, 16)	natural tree	rec 2 means	kd tree	hier (8 vertices)	hier (36 vertices)	h = 10
memory MB	...	**7**	29.28	14	7.7	
memory %	...	7.2	29.9	15	7.9	
max rank	>500	125	496	**31**	100	
compression	...	2.00E-05	7.00E-04	**5.00E-07**	6.00E-06	

ABALONE (4177, 8)	natural tree	rec 2 means	kd tree	hier (6 vertices)		h = 2
memory MB	6	**1.31**	1.36	30		
memory %	4.3	1	1	21		
max rank	98	41	**36**	45		
compression	1.40E-04	**1.00E-04**	**1.00E-04**	6.00E-05		

LETTER (5000, 16)	natural tree	rec 2 means	kd tree	hier (8 vertices)	hier (30 vertices)	h = 0.5
memory MB	22.4	**2**	2.8	30	8	
memory %	11.2	1	1.4	15	4	
max rank	432	96	94	39	**20**	
compression	3.60E-04	1.50E-06	1.90E-06	**1.90E-08**	4.00E-08	

ABALONE_normed (4177, 8)	natural tree	rec 2 means	kd tree	hier (7 vertices)		h = 0.05
memory MB	10.9	**1.223**	1.952	25.21		
memory %	7.8	0.876	1.399	18.061		
max rank	370	39	79	**36**		
compression	5.00E-05	2.16E-06	3.20E-06	**3.46E-08**		

Figure 6. Number in bold is the best achieved result over the methods tried, h is gaussian width, see below for the description of rows/columns

Each separate chart on Figure 6 contains the results for some dataset. Methods were tested on the natural datasets used taken from UCI repository [8]. For example, PENDIGITS(3498, 16) means PENDIGITS dataset (collection of handwritten digits) with 3498 data points, each described by 16 features. Synthetic datasets (mixtures of several gaussians) were also used to explore influence of the presence of cluster structure to the various compression methods.

b) Trees

Columns of the charts contain the information about various HSS trees constructions (i.e. various clustering algorithms tried). Most of the methods had several variations tried, and the ones reported in the chart are those that have shown best quality. Brief description of the base clustering methods used:

- natural tree - a baseline method to compare with. No clustering happens here, we do not use any information about mutual distances, and HSS tree is a complete binary tree

- rec 2 means - recursive two means - divisive clustering method - starts with dividing all the points into two clusters using two_means algorithm, then divides each of the clusters found into two using same method, and continues splitting until certain minimal cluster size achieved
- kd tree - another divisive clustering method - based on splitting points into two parts (on each step) looking for the best splitting point for a single coordinate. We were motivated to try this method by the papers [1] and [2].
- hier - hierarchical clustering - agglomerative clustering algorithm with ward linkage function (as it creates the most size-balanced clusters), see [10] for more details.
- bottom up glueing - starts from k clusters (found by any ‘’good’’ clustering method, may be kmeans, spectral, or Birch (which is known to be good for large number of clusters), or DBSCAN, etc) and builds down-up agglomerative tree in a balanced way.

c) Evaluation metrics

Rows of the charts on Figure 6 contain information about quality of the method obtained. Note that the metrics used are not the standard metrics measuring clustering quality, but rather specific for our compression task:

- Memory — sum of sizes of blocks used in compressed structure
- Memory % - Memory MB, normalized by the size of the matrix K ($=n^2$)
- Maximal rank — rank of the maximal dense block used in HSS-structure
- Compression quality — shows the difference between K original kernel matrix and its compressed version HSS(K), normalized by the original norm of K: $\|K - HSS(K)\|_F / \|K\|_F$ (F - Frobenius norm of the matrix, see [6]).

d) Some conclusions and further directions

Here are some empirical observations comparing the methods used.

Natural tree was the baseline for comparison (no effort for looking at cluster structure is made) - and it shows worst results indeed.

Recursive two means seems to be the best in sense of memory usage and most optimal overall.

In case of strong cluster structure in data, agglomerative methods (*hierarchical and bottom-up glueing*) give very low rank, due to finding real clusters.

Hierarchical clustering looks to be best in terms of compression quality (by an order).

When one coordinate is considerable more spread than the others, *kd-tree* works really well.

So, recursive two means divisive clustering seems to be the best preprocessing candidate so far (as the one consistently minimizing memory used: 2 to 10 times better than no-preprocessing version over the various applications). This version is implemented on C++ as well and its success is confirmed on the ‘’full-speed’’ version of the algorithm. One of the drawbacks of the recursive two means preprocessing is a relative instability (coming from the fact that kmeans depends on a random sampling on each of the recursive steps).

Some future directions here are to optimize recursive two means in C++ version with MPI and improve its stability. Another potential idea for the future work is to combine the ability of the agglomerative algorithms to find real clusters in data (when they exist) with the ability of divisive algorithms to create a splitting with low memory, and to create a hybrid preprocessing algorithms combining both these advantages.

1-4. Kernel Ridge Regression with STRUMPACK

The task of speeding up kernel matrix computations is a known problem in machine learning research, and there does not seem to be one best solution suggested up to now. There are another algorithms suggested for the kernel matrices compression (see [4] and [5]), and in [4] the authors show the results

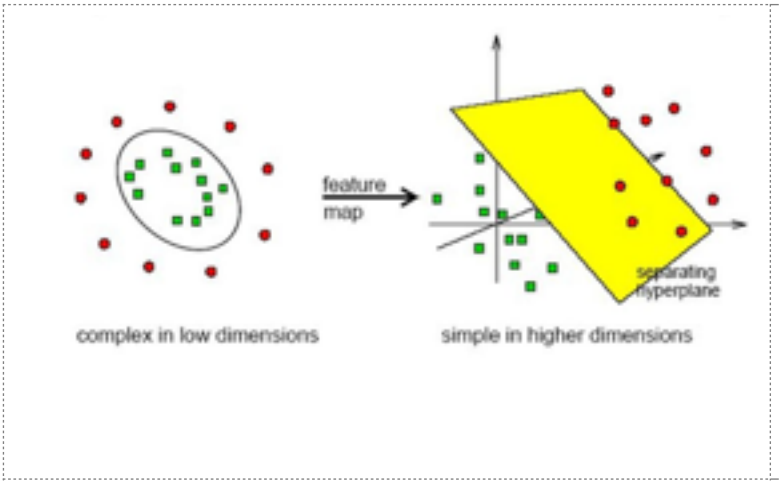

Figure 5. Ridge regression for binary classification of the data points

of the binary classification by the kernel ridge regression (see [7] and Figure 5). This is an ongoing second part of the project to fully compare classification accuracy and speed requirements made with the help of STRUMPACK with the method described in [4]. So far I did the check on Matlab for the subset of data (preserving the ratio between train and test size that the authors used) and got very promising results: I reproduced the reported accuracy for the SUSY dataset and got 100% accuracy on handwritten digits recognition dataset, as well as good compression quality.

1-5. Project results

On my opinion, the most interesting features of my project are

- STRUMPACK was tried and has shown itself useful for a completely new area of machine learning applications, and
- the idea to compare several relatively sophisticated ways to preprocess kernel matrix (i.e. cluster points in the dataset) significantly improves compression (in all the literature we have seen the authors used either natural or kd-tree and did not perform comparison of the techniques).

My role was to find and choose preprocessing clustering algorithms, as well as to implement them and compare the quality obtained, and then to test compressed matrix in the classification problem. I have discussed my ideas and the results approximately once a week with Pieter Ghysels and Sherry Li, and got valuable technical advice, as well as interesting strategical discussions.

I have presented my results in the one hour seminar for the whole group, presentation is available, see [13]. We are planning to prepare a publication based on the results obtained: comparison of various preprocessing techniques for STRUMPACK for kernel matrices, and effective kernel methods classification with STRUMPACK.

2. Reading seminars in machine learning

Through the whole 10 weeks of my internship twice a week I led a seminar in machine learning. It was based on the notes from Stanford machine learning class [9] and organized in a way that I am speaking about the slides, answering questions of the team, and we decide together what areas need to be

discussed in more details. In particular, I made a separate presentation about “unsupervised learning methods beyond PCA and clustering algorithms” (about self-organizing maps, ICA, and multidimensional scaling), and another presentation about machine learning methods in genetic studies, and a note about ridge and lasso regression usage. These materials were distributed within the team and I am planning to post them on my phd student website for the future reference. We have also extensively used the books by T. Hastie, R. Tibshirani et al and also discussed several state-of-the-art machine learning articles.

As one of the continuations of this educational activity, we now participate in a data analysis competition (which is the part of Data Science day at LBNL September 20) with the other post-docs from the team. We try to understand and predict waiting times for the computing jobs sent to NERSC (National Energy Research Scientific Computing Center at LBNL). I am happy that people are willing to apply in practice theoretical concepts we have discussed over the Summer, and I am also like that machine learning techniques can actually help researchers at LBNL better plan their interaction with the cluster.

2. Acknowledgments

I would like to thank my mentor Xiaoye Sherry Li for the chance to be involved in the research process of her wonderful team and always challenged in a positive way. I am also grateful to Pieter Ghysels for many helpful and inspiring discussions of my project, and his help with any technical question I encountered. I am thankful for the whole group that made possible the machine learning seminar and helped me feel included and motivated for the whole time of the internship. I am thankful to NSF, ORISE and LBNL who created this research opportunity and gave me the chance to participate. Finally, I am thankful to my PhD advisor Roman Vershynin who have shown me this internship opportunity in the first place and was very supportive during all the application and organization process.

3. References

[1] F.-H. Rouet, Xiaoye S. Li, P. Ghysels, A. Napov. A distributed-memory package for dense hierarchically semi-separable matrix computations using randomization. (ACM Transactions on Mathematical Software, Vol. 42, No. 4, Article 27, Publication date: June 2016.)

[2] P. G. Martinsson A fast randomized algorithm for computing a hierarchically semi separable representation of a matrix. (SIAM J. MATRIX ANAL. APPL. c 2011 Vol. 32, No. 4, pp. 1251–1274)

[3] For more information about STRUMPACK visit http://portal.nersc.gov/project/sparse/strumpack/

[4] Chenhan D. Yu, William B. March, George Biros, An NlogN Parallel Fast Direct Solver for Kernel Matrices (Preprint: arXiv:1701.02324, 2017)

[5] Ruoxi Wang, Yingzhou Li, Michael W. Mahoney, Eric Darve, Structured Block Basis Factorization for Scalable Kernel Matrix Evaluation (Technical Report, Preprint: arXiv:1502.03571, 2015)

[6] Frobenius norm of the matrix http://mathworld.wolfram.com/FrobeniusNorm.html

[7] Kernel Ridge Regression https://www.ics.uci.edu/~welling/classnotes/papers_class/Kernel-Ridge.pdf

[8] UCI Machine Learning repository http://archive.ics.uci.edu/ml/index.php

[9] Stanford CME 250 class https://sites.google.com/site/cme250winter2016/

[10] T. Hastie, R. Tibshirani, J. Friedman, The Elements of Statistical learning https://www.stanford.edu/~hastie/Papers/ESLII.pdf

[11] G. James, D. Witten, T. Hastie, R. Tibshirani An Introduction to Statistical learning http://www-bcf.usc.edu/~gareth/ISL/

[12] More on hierarchical clustering https://nlp.stanford.edu/IR-book/html/htmledition/hierarchical-agglomerative-clustering-1.html

[13] My project presentation https://drive.google.com/open?id=0B1XA_jelKGs9bGRuQTRaaW5kN3M